CMPS 202 – Lab 10 Solution

Sets and Maps

Exercise 1

```java
import java.util.*;

public class Exercise1 {

    public static Set<String> uniqueNames(ArrayList<String> names) {
        return new HashSet<>(names);
    }

    public static Set<String> commonNames(ArrayList<String> list1, ArrayList<String> list2) {
        Set<String> result = new HashSet<>(list1);
        result.retainAll(list2);
        return result;
    }

    public static Set<String> onlyInFirst(ArrayList<String> list1, ArrayList<String> list2) {
        Set<String> result = new HashSet<>(list1);
        result.removeAll(list2);
        return result;
    }

    public static void main(String[] args) {
        ArrayList<String> list1 = new ArrayList<>(Arrays.asList(
            "Ali", "Sara", "John", "Ali", "Maya", "John", "Lea"
        ));

        ArrayList<String> list2 = new ArrayList<>(Arrays.asList(
            "John", "Mira", "Sara", "Sara", "Omar", "Lea"
        ));

        System.out.println(uniqueNames(list1));
        System.out.println(commonNames(list1, list2));
        System.out.println(onlyInFirst(list1, list2));
    }
}
```

Exercise 2

```java
import java.io.*;
import java.util.*;

public class Exercise2 {

    public static Map<String, Integer> buildWordCount(Scanner input) {
        Map<String, Integer> map = new HashMap<>();

        while (input.hasNext()) {
            String word = input.next();
            if (map.containsKey(word)) {
                map.put(word, map.get(word) + 1);
            } else {
                map.put(word, 1);
            }
        }

        return map;
    }

    public static void printRepeatedWords(Map<String, Integer> map) {
        for (String word : map.keySet()) {
            if (map.get(word) > 1) {
                System.out.println(word);
            }
        }
    }

    public static String mostFrequentWord(Map<String, Integer> map) {
        String answer = null;
        int max = 0;

        for (String word : map.keySet()) {
            if (map.get(word) > max) {
                max = map.get(word);
                answer = word;
            }
        }

        return answer;
    }

    public static int totalDistinctWords(Map<String, Integer> map) {
        return map.size();
```

```
45      }
46
47      public static void main(String[] args) throws
            FileNotFoundException {
48          Scanner input = new Scanner(new File("words.txt"));
49
50          Map<String, Integer> wordCount = buildWordCount(input);
51
52          System.out.println(wordCount);
53          printRepeatedWords(wordCount);
54          System.out.println(mostFrequentWord(wordCount));
55          System.out.println(totalDistinctWords(wordCount));
56      }
57  }
```

Exercise 3

```
1   import java.util.*;
2
3   public class Exercise3 {
4
5       public static Map<Integer, Set<String>> reverseGrades(Map<
            String, Integer> studentGrades) {
6           Map<Integer, Set<String>> result = new HashMap<>();
7
8           for (String name : studentGrades.keySet()) {
9               int grade = studentGrades.get(name);
10
11              if (!result.containsKey(grade)) {
12                  result.put(grade, new HashSet<>());
13              }
14
15              result.get(grade).add(name);
16          }
17
18          return result;
19      }
20
21      public static int gradeWithMostStudents(Map<Integer, Set<
            String>> gradeMap) {
22          int bestGrade = 0;
23          int maxCount = 0;
24
25          for (Integer grade : gradeMap.keySet()) {
26              int count = gradeMap.get(grade).size();
27              if (count > maxCount) {
28                  maxCount = count;
29                  bestGrade = grade;
```

```
            }
        }

        return bestGrade;
    }

    public static Set<String> studentsWithSameGradeInBothExams(
            Map<String, Integer> exam1,
            Map<String, Integer> exam2) {

        Set<String> result = new HashSet<>();

        for (String name : exam1.keySet()) {
            if (exam2.containsKey(name) && exam1.get(name).equals
               (exam2.get(name))) {
                result.add(name);
            }
        }

        return result;
    }

    public static Map<Integer, Integer> gradeFrequencies(Map<
       Integer, Set<String>> gradeMap) {
        Map<Integer, Integer> freqMap = new HashMap<>();

        for (Integer grade : gradeMap.keySet()) {
            freqMap.put(grade, gradeMap.get(grade).size());
        }

        return freqMap;
    }

    public static Set<Integer> gradesWithAtLeastKStudents(Map<
       Integer, Integer> freqMap, int k) {
        Set<Integer> result = new HashSet<>();

        for (Integer grade : freqMap.keySet()) {
            if (freqMap.get(grade) >= k) {
                result.add(grade);
            }
        }

        return result;
    }

    public static void main(String[] args) {
        Map<String, Integer> studentGrades = new HashMap<>();
        studentGrades.put("Ali", 90);
        studentGrades.put("Sara", 85);
        studentGrades.put("John", 90);
```

```
78          studentGrades.put("Maya", 85);
79          studentGrades.put("Omar", 70);
80          studentGrades.put("Lea", 90);
81
82          Map<Integer, Set<String>> reversed = reverseGrades(
                studentGrades);
83          System.out.println(reversed);
84          System.out.println(gradeWithMostStudents(reversed));
85
86          Map<Integer, Integer> freqMap = gradeFrequencies(reversed
                );
87          System.out.println(freqMap);
88          System.out.println(gradesWithAtLeastKStudents(freqMap, 2)
                );
89
90          Map<String, Integer> exam1 = new HashMap<>();
91          exam1.put("Ali", 90);
92          exam1.put("Sara", 85);
93          exam1.put("John", 90);
94          exam1.put("Maya", 70);
95
96          Map<String, Integer> exam2 = new HashMap<>();
97          exam2.put("Ali", 90);
98          exam2.put("Sara", 80);
99          exam2.put("John", 90);
100         exam2.put("Omar", 70);
101
102         System.out.println(studentsWithSameGradeInBothExams(exam1
                , exam2));
103     }
104 }
```

Exercise 4

Part A

- The size of the set is **4**, since duplicate values are stored only once.
- The order is not fixed because `HashSet` does not preserve insertion order.
- If `HashSet` is replaced with `TreeSet`, the elements are stored in sorted order.

Part B

```
1  import java.util.*;
2
3  public class Exercise4 {
4
5      public static void removeFailing(Set<Integer> scores) {
```

```
6           Iterator<Integer> it = scores.iterator();
7           while (it.hasNext()) {
8               int x = it.next();
9               if (x < 60) {
10                  it.remove();
11              }
12          }
13      }
14
15      public static void removeFailingStudents(Map<String, Integer>
            scores) {
16          Iterator<String> it = scores.keySet().iterator();
17          while (it.hasNext()) {
18              String name = it.next();
19              int grade = scores.get(name);
20              if (grade < 60) {
21                  it.remove();
22              }
23          }
24      }
25
26      public static void main(String[] args) {
27          Set<Integer> setScores = new HashSet<>(Arrays.asList(94,
                38, 87, 43, 72));
28          removeFailing(setScores);
29          System.out.println(setScores);
30
31          Map<String, Integer> mapScores = new HashMap<>();
32          mapScores.put("Jenny", 38);
33          mapScores.put("Stef", 94);
34          mapScores.put("Greg", 87);
35          mapScores.put("Marty", 43);
36          mapScores.put("Angela", 72);
37
38          removeFailingStudents(mapScores);
39          System.out.println(mapScores);
40      }
41  }
```